Professionelle Arbeitsweisen zur effizienten Automation



 © Christian Henke 2026

Referent: Christian Henke

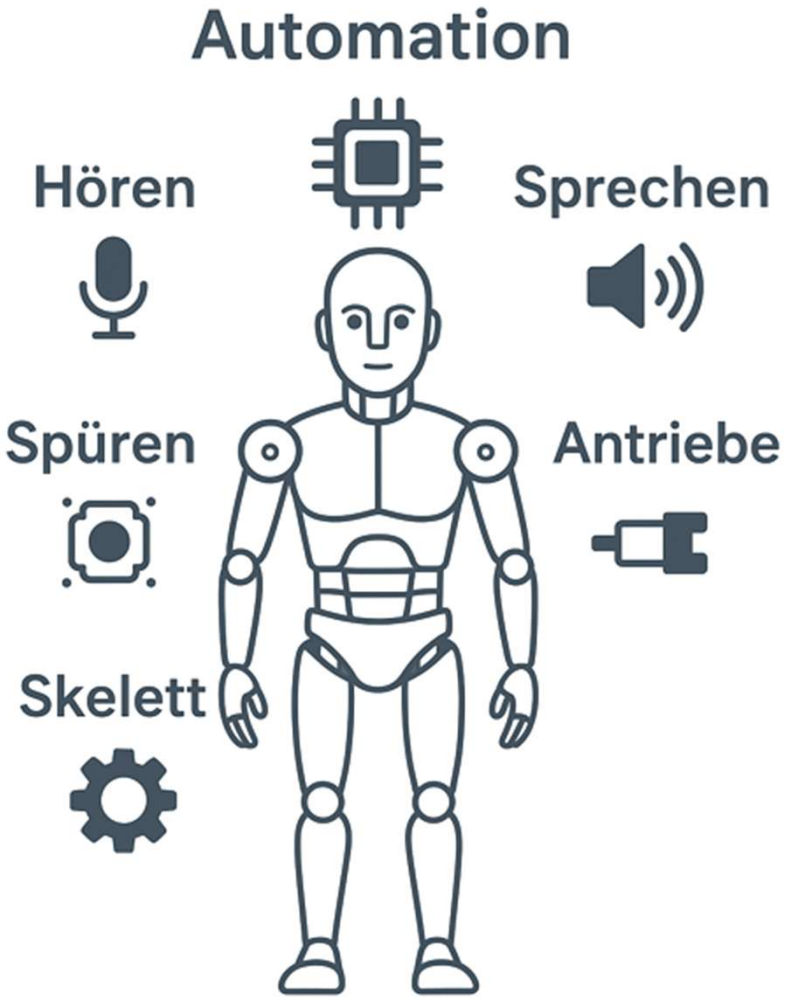
- Lehre: Industrieelektroniker
- Studium: Energie- und Automationstechnik
  - Fachhochschule Schmalkalden
  - Universität Paderborn
- Beruf(ung): 20 Jahre Dipl.-Ing. (FH) für Automatisierungstechnik bei BECKHOFF
- Jahrgang 1981



 © Christian Henke 2026

Stoff für die Zukunft

- Basis-Regeln der modernen Automatisierung
- Software Architektur für Echtzeit-System
- Safety, Security und Verfügbarkeit
- Messtechnik
- Motion Systeme
  - Rotative
  - Lineare
  - Planare
- HMI ist mehr als nur Bedienen und Anzeigen



 © Christian Henke 2026

Erwartungen

- Respekt, Hilfsbereitschaft und Wertschätzung untereinander
- Aktive Diskussionen und Mitarbeit im Forum
  - Inhaltliche Fragen zur Vorlesung werden im Forum beantwortet.
- Positive Projektarbeit
- Innerhalb des Skripts/Präsentation sind prüfungsrelevante Themen wie folgt gekennzeichnet:

Assignment

 © Christian Henke 2026

Projektarbeit

Assignment

- **Teil 1: Theoretische schriftliche Projektbeschreibung**
  Die Themen der Vorlesungsunterlagen sollen betrachtet werden.
- **Teil 2: Praktische Umsetzung in TwinCAT (TwinCAT Solution)**
  Verinnerlichen Sie die Grundregeln
- **Teil 3: Pitch der Lösung**

Details unter:

https://sakai.mci4me.at/portal/site/special.project.hoehere_automatisierungstechnik/tool/d5fdd978-486f-4d2b-887a-0585b9ca09df

 © Christian Henke 2026

Ziel des Semesters

Ziel ist das Kennenlernen von Werkzeugen und
das Erlernen moderner Technologien
zur **Entwicklung und Umsetzung**
effizienter, leistungsfähiger
Automatisierungslösungen
über alle Phasen

von der Idee über die Planung bis zum Betrieb.

 © Christian Henke 2026

Wo finden Automationslösungen?

- Energiewirtschaft
- Entertainment
- Immobilienwirtschaft (Raumautomation, Filial-Management)
- Infrastruktur
- Maschinen und Anlagen
- Medizin-Technik (Labore, Roboter, Assistenzsysteme, …)
- Mobilität (Nutzfahrzeuge, Ladeinfrastruktur, Schiffe, Raumfahrt, …)
- uvm.

 © Christian Henke 2026

Warum benötigen wir Automationslösungen?

- Automationslösungen stellen einen zentralen Baustein für die **Wettbewerbsfähigkeit** moderner Industrie- und Dienstleistungsunternehmen dar.
- Sie dienen dazu, **Prozesse effizienter, konsistenter und ressourcenschonender** zu gestalten. Durch die Automatisierung repetitiver oder fehleranfälliger Tätigkeiten können **Qualitätsschwankungen reduziert, Produktionsgeschwindigkeiten erhöht** und **Personalressourcen** gezielt für wertschöpfende Aufgaben eingesetzt werden.
- Darüber hinaus ermöglichen Automationssysteme eine kontinuierliche Datenerfassung, die als Grundlage für Analysen, Optimierungen und datenbasierte Entscheidungen während des Betriebs dient.

 © Christian Henke 2026

Warum planen wir digital?

- Ein wesentlicher Erfolgsfaktor liegt heute jedoch nicht allein in der Automatisierung selbst, sondern in der **durchgängigen digitalen Prozesskette** – also im Übergang von der Idee direkt in eine **digitale, medienbruchfreie Planung.**
- Dieser Ansatz erlaubt es, sämtliche Projektphasen – **von der Konzeption** über die Konstruktion und Simulation bis **hin zur Inbetriebnahme** – auf einer gemeinsamen Datenbasis abzubilden.
- Dadurch werden Informationsverluste vermieden, **Entwicklungszeiten signifikant verkürzt** und Änderungsprozesse transparenter gestaltet. Die digitale Durchgängigkeit bildet somit die Voraussetzung für eine nachhaltige, agile und zukunftsfähige Automatisierung.

 © Christian Henke 2026

Professionelle Arbeitsweisen zur effizienten Automation



1 Technologien und Anforderungen an die professionelle Applikations-umsetzung und Programmierung

2 Objektorientierte Programmierung nach IEC61131-3 3rd Edition

3 Motion und Safety und Kommunikationtechnologien

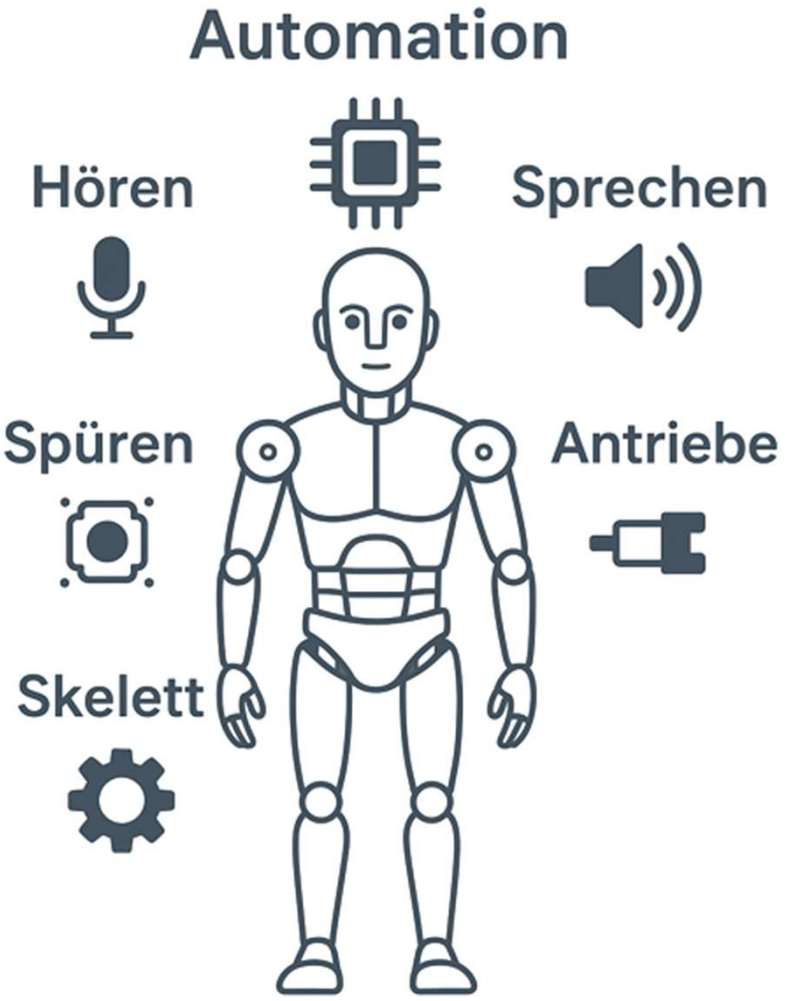
4 Humane-Maschine-Interface - HMI

 © Christian Henke 2026

Der iterative Prozess

1. Aus einer Idee wird der Planungsprozess angestoßen.
2. Innerhalb der Planung wird die Idee modularisiert, strukturiert. Jedes Modul bekommt einen Namen.
3. Jede Änderung eines Moduls wird typischerweise versioniert. Das gilt für alle Module in allen Phasen des Entstehungsprozesses
4. Vor der realen Umsetzung folgt die wirtschaftliche Betrachtung und die Validierung der Planung.

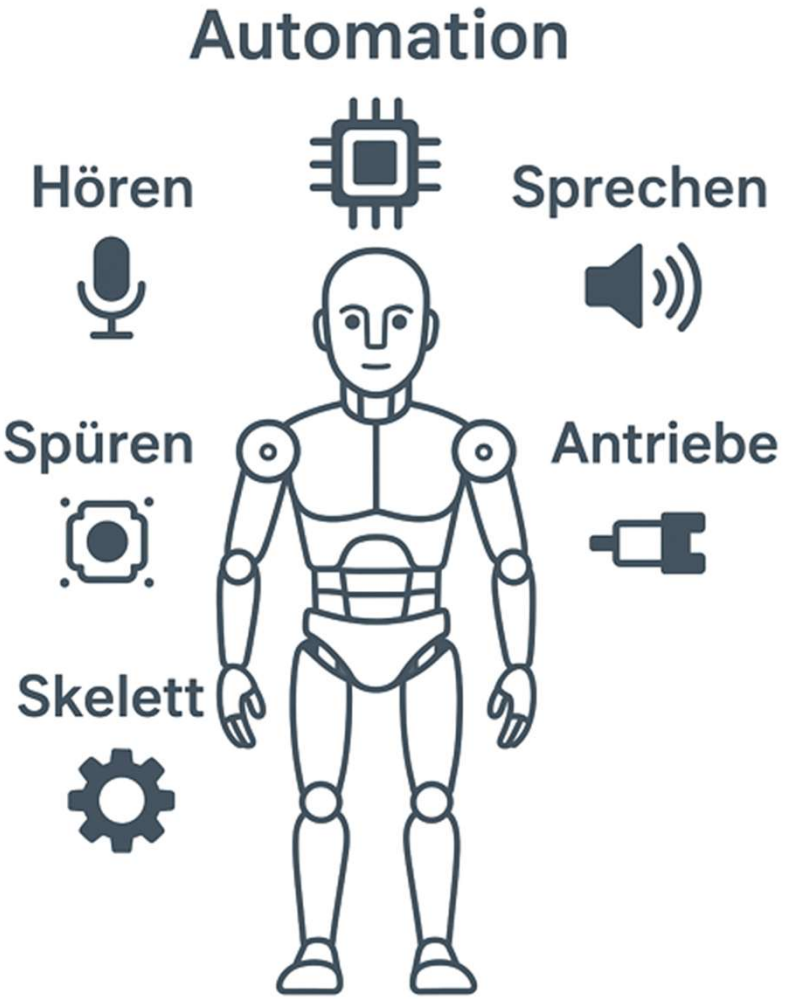
- Zwischen den Schritten kann mehrfach gewechselt werden.



 © Christian Henke 2026

Der iterative Prozess

5. Auch die technische Umsetzung wird modularisiert, strukturiert. Jedes Modul übernimmt seinen Namen aus der Planung.
6. Nach der Prototypenphase beginnt die Optimierung der Umsetzungs- und Herstellungsprozesse. Mit Hilfe von Skripten und Fernsteuerung von Engineering-Werkzeugen werden die Herstellungsosten gesenkt.



- Zwischen den Schritten kann mehrfach gewechselt werden.

 © Christian Henke 2026

Digitaler Clone – Industrie 4.0 – BIM – PLM

- Was ist ein Digitaler Clone?
  - Man spricht von einem Planungsmodell, welches abschließend in ein As-Build-Modell für den Betrieb überführt wird.

- Welche Zielgrößen hat ein Digitaler Clone:
  - Einheitliche Datenmodelle
  - Nachvollziehbarer Informationsfluss
  - Verhindern von Redundanzen und Medienbrüchen
  - Sicherstellung der Datenqualität

- Was benötigt man für ein erfolgreiches Automationsprojekt?
  - System-Architekt für angewandte Software mit Kompetenzen und Verantwortung.

 © Christian Henke 2026

Digitaler Clone – Industrie 4.0 – BIM – PLM

- Bereiche die der System-Architekt im Fokus hat

  - **Planung** (z. B. CAD, E-CAD, PDM, PLM)
  - **Entwicklung** (z. B. SPS, Software, Simulation)
  - **Produktion** (z. B. MES, ERP)
  - **Betrieb** (z. B. SCADA, Instandhaltung, CMMS, Facility-Management)

- Ziel ist die **Definition eines einheitlichen, durchgängigen Datenflusses** zwischen den beteiligten Gewerken.

 © Christian Henke 2026

Digitaler Clone – Industrie 4.0 – BIM – PLM

- Durch die richtigen Fragen und Aufgaben erfüllt der System-Architekt das Ziel?

  - Welche **Standards/Normen** werden verwendet?
    - HINWEIS: Eine Norm beschreibt in der Regel den kleinsten Kompromiss verschiedener Hersteller in einem Gremium. Es ist hilfreich neben der Norm weitere bessere Qualitätskriterien zu definieren.
  - Welche Datenaustauschformate (**Dateiformate**, **Protokolle**) müssen verwendet werden?
  - Definieren welche Daten von welchen **Personen** zu welcher **Zeit** geliefert werden müssen.
  - Validierung/Bereitstellung von Tools zur **Validierung** großer Datenmengen (Prüftool).

 © Christian Henke 2026

Leitregeln für gute Lösungen!

Assignment

Die Automationslösung muss **effizienter** als bisherige Lösungen sein!

Die Regeln gelten für alle Aufgabe der Planung und Programmierung.

Vermeide unnötige Mausklicks und Eingaben!

1. Gute Lösungen sind einfach.
2. Erfinde nichts, was es schon gibt.
3. Setze auf Technologie-Standards statt Hersteller-Standards.

 © Christian Henke 2026

Leitregeln für gute Lösungen!

Assignment

- Hardware
  1. Betrachte die Wirkungsgrade.
     Mechanik und Pneumatik durch Elektrische Systeme ersetzen.
  2. Verwende nachhaltige Hardware (Lieferbarkeit).
  3. Achten Sie darauf, dass alle Geräte wartungszugänglich sind.

 © Christian Henke 2026

Leitregeln für gute Lösungen!

Assignment

- Software
  1. Namensgebung bzw. Betriebsmittelkennzeichnung objektorientiert definieren.
  2. Einheiten von physikalischen Werten definieren.
  3. Halte keine Daten doppelt (Ausnahme Redundanz-Systeme).
  4. Programmiere keine Logik doppelt.

     CTRL+C > CTRL+V / Copy Paste ist verboten

     Statt zu kopieren, referenziert man Daten und Code!
  5. Die Applikationslogik hat keine internen Abhängigkeiten zu einem herstellerspezifischen Produkt.
  6. Für jedes physische Produkt einer Automationslösung gibt es genau einen Funktionsbaustein.

 © Christian Henke 2026

Planung eine Automationslösung

In der Planung
einer Automationslösung werden
wirtschaftliche Aspekte
und anschließend
technische Aspekte betrachtet.

 © Christian Henke 2026

Marktvorgaben

- **Der maximale Preis ist vorgegeben:**
  Es wird versucht die beste Funktionalität zu finden.

- **Funktion und Qualitätskriterien sind vorgegeben:**
  Es wird versucht den besten Preis zu finden.
  Hier müssen der Verkaufspreis und die laufenden Kosten betrachtet werden.

 © Christian Henke 2026

Investitions- und Betriebskosten

- **CAPEX (Capital Expenditures):** Anschaffung, Planung, Engineering, Montage, Inbetriebnahme.
- **OPEX (Operational Expenditures):** Energie, Wartung, Personal, Ersatzteile, Stillstandskosten.
- **Total Cost of Ownership (TCO):** Gesamtbetrachtung über die Lebensdauer.

 © Christian Henke 2026

Wirtschaftlichkeitsanalyse

- **Return on Investment (ROI)** und **Amortisationszeit** als zentrale Kennzahlen.

- **Produktionsvolumen vs. Flexibilität:** Abwägung zwischen Standardisierung und Anpassungsfähigkeit.

- **Automatisierungsgrad vs. Kostenstruktur:** Höhere Automatisierung senkt Personalkosten, erhöht aber Kapitalbindung.

 © Christian Henke 2026

Risiko- und Projektmanagement

- **Technologische Risiken:** Fehlende Reife neuer Technologien, Schnittstellenprobleme.
- **Lieferketten- und Beschaffungsrisiken:** Verfügbarkeit von Komponenten, Lieferzeiten.
- **Zeit- und Kostenrisiken:** Pufferplanung, Änderungsmanagement, Nachverfolgbarkeit.

 © Christian Henke 2026

Planung eine Automationslösung

Wenn die wirtschaftlichen Kennzahlen einen Erfolg versprechen, folgt die technische Betrachtung.

Oft werden während der wirtschaftlichen Betrachtung erst grobe Schätzung der technischen Lösung benötigt.

 © Christian Henke 2026

Anforderungsanalyse und Systemdefinition

- **Lasten- und Pflichtenheft**: Klare Definition von Funktion, Leistung, Qualität, Taktzeit, Flexibilität, Sicherheitsanforderungen, Energiebedarf.

- **Schnittstellenanalyse**: Mechanische, elektrische, pneumatische und digitale Schnittstellen zwischen Teilsystemen und externen Anlagen.

- **Verfügbarkeitsgrad**: Festlegung des optimalen Verschließ, Wartungsintervall, Redundanz, Service-Intervall, Bereitschaftsdienste, …

 © Christian Henke 2026

Konstruktion und Systemarchitektur

- **Mechanisches Design**: Auswahl geeigneter Werkstoffe, Antriebstechnologien, Bewegungsmechanismen und Strukturfestigkeit.

- **Modularität und Skalierbarkeit**: Gestaltung modularer Baugruppen zur Wiederverwendbarkeit und Anpassbarkeit zukünftiger Anlagen.

- **Energieeffizienz und Nachhaltigkeit**: Minimierung von Energieverlusten, Einsatz energieeffizienter (Antriebs-)Systeme und Rekuperationssysteme.

 © Christian Henke 2026

Automatisierung und Steuerung

- **Steuerungsarchitektur**: Wahl der Steuerungsplattform (z. B. Beckhoff TwinCAT, Siemens TIA Portal) und Kommunikationsinfrastruktur (z. B. EtherCAT, PROFINET, OPC UA).
- **Sicherheitskonzept**: Integration funktionaler Sicherheit (SIL, PL), Not-Aus-Systeme, sichere Kommunikation.
- **Datenmanagement und Digitalisierung**: Einbindung von Sensorik, Zustandsüberwachung, Predictive Maintenance und MES-/ERP-Schnittstellen.
- **Software-Engineering**: Strukturierte, modulare Programmierung nach IEC 61131-3 und objektorientierten Methoden, Versionierung und Simulation.

 © Christian Henke 2026

Validierung und Betrieb

- **Simulation und virtuelle Inbetriebnahme** zur Reduzierung von Testzeiten und Inbetriebnahmerisiken.
- **Inbetriebnahme, Dokumentation, Schulung** für Bedien- und Wartungspersonal.
- **Lebenszyklusbetrachtung**: Wartbarkeit, Ersatzteilversorgung, Retrofit-Fähigkeit.

 © Christian Henke 2026

Technische und funktionale Validierung

- **Lasten- und Pflichtenheft**
  - **Vollständigkeit und Eindeutigkeit prüfen**: Sind alle Funktionen, Betriebsarten, Schnittstellen und Sicherheitsanforderungen klar beschrieben?
  - **Akzeptanzkriterien definiert?** (z. B. Genauigkeiten, Reaktionszeiten, Energieverbrauch, Frontend-Design)
  - **Grenzfälle und Sonderzustände** festgelegt (z. B. Not-Aus, Fehlerreaktionen, Wiederanlauf)?

 © Christian Henke 2026

Validierung

- **Funktionsumfang und Schnittstellen**
  - Validierung aller **Ein- und Ausgänge**, Bussysteme und Kommunikationsschnittstellen (z. B. EtherCAT, PROFINET, OPC UA, MQTT).
  - **Schnittstellen zwischen Maschinen, Stationen oder übergeordneten Systemen** (z. B. MES, ERP, Robotersteuerung) eindeutig beschrieben und getestet?
    - Tipp: Wenn eine neue Schnittstelle in der Unternehmung zur Anwendung kommt, testen Sie diese prototypisch in den eigenen vier Wänden.

 © Christian Henke 2026

Mechanische und elektrische Voraussetzungen

- **Mechanik**
  - Sind **Bewegungsrichtungen, Endlagen, Sensorpositionen** und **Antriebsauslegungen** definiert und dokumentiert?
  - **Virtuelles Modell (z. B. CAD oder Digital Twin)** vorhanden und mit den Bewegungsdaten abgeglichen?
  - **Kollisionsfreiheit** in allen Bewegungsabläufen validiert?

 © Christian Henke 2026

Validierung

- **Elektrik / Steuerungstechnik**
  - **Schaltpläne, Klemmenpläne und SPS-Hardwarekonfiguration** freigegeben?
  - **Adressierung** (Netzwerke, IPs, Steuerungsinstanzen) eindeutig?

- **Versorgungskonzept** (24 V DC, 230 V AC , 400 V AC, Zwischenkreise, Rückspeisung) vollständig definiert?

- **Sicherheitskreise (SIL/PL)** technisch geprüft und dokumentiert?

  Hinweis: Um grobe Fahrlässigkeit auszuschließen, lassen Sie sich in den einzusetzenden Produkten und in Maschinenverordnung zertifiziert ausbilden.

 © Christian Henke 2026

Wirtschaftliche und organisatorische Validierung

- **Zeitplan und Abhängigkeiten** zwischen Mechanik, Elektrik, Software abgestimmt?
- **Verfügbarkeit aller Komponenten** (Lieferzeiten, Firmwarestände, Lizenzen, Engineering-Software, IT-Security-Freigaben) geprüft?
- **Personalressourcen und Zuständigkeiten** (Programmierer, Inbetriebnehmer, Safety-Experten) festgelegt?
- **Änderungsmanagement** vereinbart (Versionierung, Änderungsfreigaben)?

 © Christian Henke 2026

Dokumentation und Freigabe

- **Versionsstände aller Unterlagen** (Lastenheft, Pflichtenheft, E-Pläne, CAD) freigegeben und eindeutig?
- **Risikoanalyse**
  - nach Maschinenverordnung abgeschlossen?
  - CRA bzw. NIS2 Anforderungen erfüllbar?

- **Softwarefreigabe zur Implementierung** durch Projektleitung oder Kunde erfolgt?

 © Christian Henke 2026

Validierung

- Nach dem nun alle theoretischen Betrachtungen erfüllt sind, kann es mit der Umsetzung der Software los gehen.

- Zerlegen Sie die Anlage/Maschine/Applikation in

 © Christian Henke 2026

Welche Vorteile bringt die Modularisierung?

- Komplexität reduzieren,
- Wiederverwendbarkeit erhöhen,
- Entwicklungszeit, Testzeit und Risiko senken.

 © Christian Henke 2026

Vorgehensweise?

1. Analysephase – Gemeinsamkeiten und Unterschiede erkennen
2. Funktions- und Modulstruktur aufbauen
3. Softwareseitige Modularisierung
4. Standardisierung und Wiederverwendung
5. Integration und Parametrierung
6. Validierung und Dokumentation

 © Christian Henke 2026

Analysephase

- **Ziel:** Identifizieren, welche Teile *gleichartig*, *ähnlich* oder *spezifisch* sind.

- **Vorgehen (1/2)**
  - Bestehende Maschinen/Anlagen analysieren:
    - **Funktionen vergleichen:** z. B. Fördern, Dosieren, Schweißen, …
    - **Automatisierungskomponenten vergleichen:** Steuerungstyp, Bussysteme, HMI, Safety, Motion.
    - **Schnittstellen und Signalstrukturen** prüfen (I/O-Baugruppen, Kommunikationsprotokolle).

 © Christian Henke 2026

Analysephase

- **Vorgehen (2/2)**
  - **Gemeinsamkeiten und Unterschiede dokumentieren** (z. B. in einer Morphologischen Matrix).

- **Ergebnis: Funktionslandkarte** oder **Baugruppenübersicht**, in der jedes Modul klar abgegrenzt ist.

 © Christian Henke 2026

Funktions- und Modulstruktur

- **Ziel:** Funktionale Strukturierung in **unabhängig testbare, wiederverwendbare Module**.

- **Vorgehen (1/2)**
  - Zerlege die Anlage in **Funktionsgruppen** oder **Module**:
    - Mechanische Module (z. B. Förderstrecke, Greifer, Werkzeugkopf)
    - Steuerungsmodule (z. B. Achssteuerung, Sensorüberwachung, Sicherheitslogik)
    - HMI-Module (z. B. Aggregate, Funktionen, Parameterseite, Alarme)

 © Christian Henke 2026

Funktions- und Modulstruktur

- **Vorgehen (2/2)**
  - Definiere für jedes Modul:
    - **Schnittstellen (Inputs/Outputs, Events, Parameter, Erwartungswerte (welcher Wertebereich ist erlaubt))**
    - **Verhalten** (Zustände, Sequenzen, Fehlermeldungen)
    - **Kommunikation** zu anderen Modulen (z. B. Start/Stopp, Freigaben, Fehlerweitergabe)

- **Ergebnis:** Eine **hierarchische Struktur** nach dem Baukastenprinzip – mechanisch, elektrisch und softwaretechnisch kompatibel.

 © Christian Henke 2026

Softwareseitige Modularisierung

- **Ziel:** Trennung von **Funktionseinheiten** in wiederverwendbare Softwareobjekte (z. B. Funktionsbausteine).

- **Vorgehen (1/2)**
  - Implementiere jeden Modul-Funktionsblock nach dem Prinzip: **Input → Verarbeitung → Output,** klar definierte Schnittstelle.
  - Verwende **objektorientierte Programmierung (OOP)** und **Bibliotheken.**

 © Christian Henke 2026

Softwareseitige Modularisierung

- **Vorgehen (2/2)**
  - Modularisiere
    - **Datenstrukturen**
    - **Funktionsblöcke** mit definiertem Verhalten
  - Entscheide, welche Daten sind
    - **global, lokal, spannungsausfallsicher**
    - **variabel oder konstant**
    - **permanent oder temporär**

 © Christian Henke 2026

Standardisierung und Wiederverwendung

- **Ziel:** Schaffung einer **Bibliothek** oder eines **Frameworks**, das projektübergreifend genutzt werden kann.

- **Vorgehen (1/2)**
  - Standardisiere:
    - **Namenskonventionen**
    - **Signal- und Variablenstruktur**
    - **Fehlerhandling & Zustandsmodelle**
    - **Kommunikationsschnittstellen**

 © Christian Henke 2026

Standardisierung und Wiederverwendung

- **Vorgehen (2/2)**
  - Dokumentiere die Module in einer **Modulbibliothek** mit:
    - Beschreibung der Funktion
    - Parameterliste
    - Testbedingungen
    - Beispielinstanz

- **Ergebnis:** Es entsteht ein **baukastenartiges Automatisierungssystem**, das mit geringem Aufwand für neue Maschinen adaptiert werden kann.

 © Christian Henke 2026

Integration und Parametrierung

- **Ziel:** Module flexibel kombinierbar machen.

- **Vorgehen (1/2)**
  - Jedes Modul muss **parametrierbar** statt fest codiert sein (z. B. Achsanzahl, Grenzwerte, Sensortypen, Schrittkettenvarianten).
  - Einheitliche **Kommunikationsschnittstellen**.

 © Christian Henke 2026

Integration und Parametrierung

- **Vorgehen (2/2)**
  - Integration über:
    - **Standardisierte Modulmanager** oder **Abarbeitungssteuerung**
    - **Automatische Modulregistrierung** (z. B. in globale Funktionsblöcke oder externe Dateien (z.B. JSON))
      - Modulmanager rufen zyklisch den Code auf und ermitteln Summenstörungen oder berechnen den aktuellen Energieverbrauch und Betriebszeiten.

 © Christian Henke 2026

Validierung und Dokumentation

- **Ziel: Qualitativ hochwertige Software**

- **Vorgehen**
  - Jedes Modul wird **eigenständig getestet** (Unit-Test / Simulation).
  - Ergebnisse dokumentieren (z. B. **UML-Charts**, Testprotokolle, Versionen, Abhängigkeiten).

- Bei der **Gesamtintegration prüfen:**
  - Konsistenz der Schnittstellen
  - Verhalten bei Modulerweiterung

 © Christian Henke 2026

Welche Vorteile bringt die Versionierung?

- Versionierung schafft **Vertrauen** in die **Wiederholbarkeit** und **Stabilität** von Softwareständen.

- Versionierung dient der **Nachvollziehbarkeit der Entwicklungshistorie**:
  - Welche Softwareversion läuft auf in welcher CPU?
  - Welche Änderungen wurden wann, von wem und warum durchgeführt?
  - Welche Version ist freigegeben, getestet oder produktiv im Einsatz?

 © Christian Henke 2026

Vorgehensweise?

1. Versionsschema definieren
2. Praktische Umsetzung im Projekt
3. Branching-Strategie
4. Dokumentation und Rückverfolgbarkeit
5. Integration in den Entwicklungsprozess

 © Christian Henke 2026

Versionierungsschema

- **Syntax** Major . Minor . Patch/Build . Revision (z.B. Version 3.2.1.0)

- Die Nummer wird **inkrementiert,**
  - **Major:** wenn die neue Version inkompatibel zur vorherigen Version ist. Dabei werden die Stellen für Minor und Build auf eins zurückgesetzt.
  - **Minor:** wenn die neue Version Funktionserweiterungen beinhaltet. Dabei wird die Stelle für Build auf eins zurückgesetzt.
  - **Patch:** wenn die neue Version nur Fehlerbehebungen beinhaltet.
  - **Revision:** wenn eine neue BETA gebaut wurde. (Sie wird bei jedem Release in produktive Systeme auf null gesetzt)

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Versionsverwaltungssystem**
  - Am besten nutzt man ein **Versionskontrollsystem (VCS)** wie:
  - **Git** (Standard, auch in TwinCAT oder Codesys integriert)
  - Alternativ: **Azure DevOps**, **GitLab**, **GitHub Enterprise, …**

Mit VCS kann man

- Änderungen rückverfolgen
- Versionen mit Tags markieren
- Branches für Entwicklungsstände anlegen

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Projektstruktur**

  – Jede Softwarekomponente (Strukturen, POUs, Bibliotheken, Package/Setups/Workloads, etc.) hat eine Versionsnummer.

  – Die automatische Vergabe vorn Versionsnummer verhindert Fehler bei der manuellen Pflege.

    Doppelt vergebene Versionsnummer, speziell in Release-Versionen führt zu massiver Verunsicherung bei den Benutzern.

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Änderungsdokumentation (Changelog)**

  - **Jede Änderung** dokumentiert man im Sourcecode. Meist als Kommentar am Anfang **jeder Datei**. Funktionsänderungen bzw. Erweiterungen in allen weiteren Dokumentationen ergänzen.
  - **Inhalt:** Versionsnummer, Text: Was wurde wann von wem geändert.

  - Changelogs werden bei Abnahmen oder in Qualitätssicherungssystemen gefordert und sind für Wartungen hilfreich.

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Branch-Strategie**

  – Um Entwicklungs- und Freigabeprozesse zu strukturieren, verwendet man ein **Branch-Modell**, z. B. **Git Flow**

    z.B. main/master, develop, feature, hotfix

  – **Regel!** Nur getestete, freigegebene Versionen dürfen auf *main/master* gemerged werden. Hieraus resultieren die Release-Version für produktive Systeme.

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Dokumentation und Rückverfolgbarkeit**
  - Was ist Inhalt der Dokumentation?
    - Versionsnummer, Datum
    - Kurzbeschreibung der Änderungen und Erweiterungen
    - Freigabestatus
    - Autor/Freigabeverantwortliche (meinst nur intern)
    - Referenzen auf Prüfdokumente (meist nur intern)

  - Dies kann in einer zentralen **Software-Freigabeliste** oder einem **Release-Register bzw. SBOM*** geführt werden.

* Software – Build Of Material

 © Christian Henke 2026

Praktische Umsetzung im Projekt

- **Integration in den Entwicklungsprozess**
  - Versionsnummer möglichst automatisch hochzählen
  - Versionen im Frontend einblenden (z.B. HMI, Steuerungsdisplay, …)

- **Test und Freigabe**
  - Vor jeder Freigabe Regressionstests oder Simulation durchführen!
  - Nur getestet Stände mit der Revision „0“ oder Tag-Release ausstatten!

 © Christian Henke 2026

Beispiel für einen Header

```
//##################################################################
//## Name/Type/Interfaces:                                        ##
//## Autor: ...                                                   ##
//## Datum: ...          Version: ... (z.B. 3.2.1.0)              ##
//## Description:                                                 ##
//## Links/readme:                                                ##
//##################################################################
```

 © Christian Henke 2026

Versionierung

- Vor der Programmierung müssen wir uns noch Gedanken über die Namensgebung für die einzelnen Modultypen und Funktionen machen.

- Aus einer professionellen Projektstruktur und Namensgebung lässt sich über 90% der Softwarestruktur und Funktionen ableiten bzw. generieren.

 © Christian Henke 2026

Anforderungen

- **Anlagenkennzeichnungssystem (AKS),**
  Betriebsmittelkennzeichnung (BMK), …

dienen dazu, **jedes technische Objekt**
eindeutig, logisch und dauerhaft zu identifizieren –

unabhängig von Hersteller, Software oder Gewerk.

 © Christian Henke 2026

Anforderungen

- Zweck der Namensgebung
  - **Eindeutige Zuordnung** von Geräten, Signalen, Datenpunkten, Schaltplänen, Dokumenten
  - **Nachvollziehbarkeit** im gesamten Engineering- und Lebenszyklus
  - **Integration über Gewerke hinweg** (Mechanik, Elektrik, Automatisierung, Gebäudeautomation)
  - **Standardisierte maschinenlesbare Schnittstellen** zu ERP, PLM, CAD, SCADA, MES usw.

 © Christian Henke 2026

Normative Grundlage

- Die wichtigsten Normen für Kennzeichnungssysteme:
  - EN ISO 81346 (Teil 1–3) : Strukturierungsprinzipien für technische Systeme, Anlagen und Produkte — definiert Objektkennzeichnung nach *Ort*, *Funktion* und *Produktaspekt*.
  - IEC 81346-10 / -12 : Anwendung im Bereich Energie- und Industrieanlagen.
  - ISO/TS 16952 : Kennzeichnung in der Kraftwerkstechnik.
  - DIN 6779 : Betriebsmittelkennzeichnung im Maschinenbau.
  - VDI 2770 : Informationsmanagement von technischen Dokumentationen, referenziert auf Kennzeichnungen.

 © Christian Henke 2026

Grundprinzip - Objektorientierte Strukturierung

Die Kennzeichnung folgt dem **Mehr-Aspekt-Prinzip!**

- **Orts-Aspekt:**
  - Wo befindet sich das Objekt?
- **Referenz-Aspekt:**
  - Mit welcher übergeordneten Einheit ist das Gerät verbunden
- **Geräte-Aspekt:**
  - Was ist das Objekt physisch?
- **Funktions-Aspekt:**
  - Was tut das Objekt?

 © Christian Henke 2026

Herausforderungen

- Die meisten Programmiersprachen sind nur folgende Sonderzeichner zulässig:
  - „_“ – erlaubtes Trennzeichen
  - „.“ – Darstellung der Hierarchie

- Die namensgebenden Normen verwenden oft weiter Sonderzeichen!
  - Hier sollte ein Kompromiss gefunden werden, damit keine zusätzlichen Software-Converter entwickelt werden müssen.

 © Christian Henke 2026

Praktische Richtlinien

- Keine Sonderzeichen (Nur Buchstaben ohne Umlaute, Zahlen und „_“
- Keine führenden Zahlen
- Lesbarkeit vor Kürze
- Hierarchie immer gleich aufbauen (Verortung, Anlage, Produkt, Funktion)
- Konsistente Abkürzungen
- CamelCase für Bezeichner
- UpperCase für Datentypen
- Hierarchische Organisation der Anlage

 © Christian Henke 2026

Notlösung

- Wenn im Projekt keine Vorgaben gemacht wurden bzw. wenn sich zwischen allen Projektbeteiligten nicht auf eine Namensgebung geeinigt werden kann, dann bietet sich folgende Notlösung:

  Verortungsschlüssel wird vom Investor oder aus der Modularisierung vorgegeben, gefolgt von einem Kürzel pro Firma, Abteilung, bzw. Technologie und wird mit der Funktion abgeschlossen.

Wichtig! Seitenzahlen, Dateinamen, Datum und andere dynamische Daten werden der Lebensdauer haben nicht im eindeutig stabilen Namen verloren!

 © Christian Henke 2026

Namensgebung – Schnittstellen-Definitionen

Organisationen

- Namensgebende Organisationen
  - OPC Foundation – UA Companion Specifications
    https://opcfoundation.org/about/opc-technologies/opc-ua/ua-companion-specifications/
  - AMEW – BACtwin
    https://www.amev-online.de/AMEVInhalt/Planen/Gebaeudeautomation/BACtwin/
  - PLCopen – Logic, Motion und Safety Funktionen
    https://www.plcopen.org/
  - usw.

 © Christian Henke 2026

Technologie zur Fernsteuerung von Engineering-Tools

- Moderne Tools bieten Application Programming Interfaces, kurz API, um externe Daten zu übertragen bzw. das Tool fernzusteuern.
  - Z.B. TwinCAT Automation Interface, ELAN API, AutoDesk Design Automation API,

- **Warum sind APIs im Engineering wichtig?**
  - Sie ermöglichen **automatisierte Workflows** (z. B. Generieren von Plänen, Berichten, Simulationsdaten)
  - Sie sorgen für **Interoperabilität** zwischen Software-Tools (z. B. CAD ↔ SPS ↔ HMI – Zu achten ist auf die Bidirektionalität!)
  - Sie sind Grundlage für **Digital Twin/Clone.**

 © Christian Henke 2026

Technologie zur Fernsteuerung von Engineering-Tools

- Sofern keine API existiert, sind Datei-basierte **maschinenlesbare** Austauschformate eine Alternative, um das Engineering zu beschleunigen.

- **AutomationML**
  - Datenaustausch zwischen CAD- und PLC-Engineering
- **BACnet EDE-Datei**
  - Datenaustausch zwischen PLC-Engineering und HMI/MBA-Engineering
- **CSV, Excel, XML, json, …**
  - Wichtig bei nicht normierten Formaten ist es, dass eine Dokumentation für den Aufbau der Datei vorliegt.

TIPP: Viele Engineering-Tools speichern ihre Daten als XML oder komprimieren diese als ZIP. Das Ändern der Dateiendung kann helfen „proprietäre“ Dateien zu öffnen.

 © Christian Henke 2026

SPS-Hardwareauslegung

Entwicklung einer methodischen Vorgehensweise zur Auslegung der SPS-Hardware unter Berücksichtigung von:

- Echtzeitanforderungen,
- Kommunikationsstrukturen,
- Sensorik und Aktorik,
- sowie zusätzlicher Systemdienste.

 © Christian Henke 2026

Überblick der Themen

1. Echtzeitanforderungen:
   - Abtastraten
   - Zykluszeiten
   - Kommunikationsraten
2. Zustandsmatrix
3. Zentrale vs. dezentrale Architektur
4. Konkurrierender Speicherzugriff
5. Zusätzliche Dienste auf der CPU.

 © Christian Henke 2026

Echtzeit-Systemgrenzen

- **Sensorik:** Datenerfassung – bestimmt notwendige Abtastraten und Signalauflösung. (Nyquist-Kriterium, Sicherheitsfaktor 3–10×)
- **Aktorik:** Prozessbeeinflussung – limitiert Reaktionszeiten und bestimmt Zyklusfrequenzen. (Dynamic, Energieverbrauch, …)
- **SPS:** zentrale oder dezentrale Verarbeitung, Priorisierung und Synchronisierung. (Jitter, Querkommunikation)
- **Kommunikation:** Datenübertragung zwischen CPU, Peripherie und Subsystemen. (Feldbustechnologie – Client-Server-Architektur oder EtherCAT)

 © Christian Henke 2026

Zustandsmatrix

- Aus den **Betriebsmatrix** lassen sich die Ein- und Ausgänge der Logik-Funktionsbausteine ableiten.

- Aus der **Havarie-Matrix** werden Maßnahmen zur Verfügbarkeit definiert. Z.B. kann man einen Aktor mittels einer Variable mit Wert TRUE über einen digitalen Ausgang einschalten oder man schließt den Aktor an einen Öffner-Kontakt eines Koppelrelais an. Hierbei wird der Aktor nun mit dem Wert FALSE aus der SPS eingeschalten.

  EtherCAT I/Os wie z.B. EL2068 bieten einen Safe-State oder Watchdog-Werte, die bei einem Ausfall der Kommunikation von Master angenommen werden.

 © Christian Henke 2026

Zustandsmatrix für Betrieb

- Zeilenweise werden alle Aktoren aufgelistet.
- Spaltenweise werden alle Sensoren bzw. dessen Messwerte aufgelistet.

- In der Betriebsmatrix wird für jeden Sensor bzw. Messwert ein Kreuz beim jeweiligen Aktor verortet, welche funktional durch den Sensor gesteuert oder geregelt wird.

 © Christian Henke 2026

Zustandsmatrix für Havarie

- Zeilenweise werden alle Aktoren aufgelistet
- Spaltenweise werden alle elektrischen Geräte aufgelistet, welche eine funktionale Verbindung mit den Aktoren aufweisen.
  - FI-Schalter, LS-Schalter, AC/DC-Netzteil, Sekundär Sicherung, SPS, Firewall, Switch, I/Os, z.B. Koppelrelais, Motorschütz, etc.

- In der Havarie-Matrix wird für jeden Aktor ein Zustand definiert, welche bei Ausfall vorausgehenden Komponente eingenommen wird.

- Sollte eine 100%tige Funktion im Havariefall gewährleistet werden müssen, dann benötigt man redundante System.

 © Christian Henke 2026

Redundantes System

- Automatische oder manuelle Umschaltung zwischen System A und B
  - Anforderung:
    - Wie lang darf die Umschaltung maximal dauern?
    - Wie werden A und B synchron gehalten?

- Welche Komponenten werden doppelt ausgeführt?
  - Sensoren, CPUs, Aktoren, Infrastruktur-Komponenten

 © Christian Henke 2026

Diversität in redundanten Systemen

- Ausfälle können zufällig oder systematisch sein.
  - **Zufällige** Ausfälle können durch einfache Verdoppelung der Hardware reduziert werden.
  - **Systematische** Fehler können in
    - Firmware, Betriebssystem oder in der Applikationsprogrammierung vorliegen.
    - Bauteilen, Produkten und Produktionsprozessen vorliegen.

  - Lösung: Diversität, um die Verfügbarkeit zu erhöhen.
    - Z.B. zwei verschiedene Steuerungsprodukte mit unterschiedlichen Betriebssystemen.

 © Christian Henke 2026

Zentral oder dezentrale Steuerungstechnik

Aspekt	Zentral	Dezentral
Rechenlast	Konzentriert (Im Gesamten weniger)	Verteilt
Kommunikationsaufwand	Geringer	Höher
Fehlerlokalisierung	Einfacher	Schwieriger
Engineering-Aufwand Update-Rollout	Geringer	Höher
Security / Safety Komplexität	Geringer	Höher

 © Christian Henke 2026

Multitask oder Multicore-Technologie

- Konkurrierender Speicherzugriff aus zwei Task auf denselben RAM möglich!

- **Typischer Konflikt:**
  - Task A schreibt auf eine globale Variable, Task B liest diese nahezu zeitgleich.

- **Lösung:**
  - Mutex
  - Semaphor

 © Christian Henke 2026

Mutex

- **Zweck:**

  Ein *Mutex* stellt sicher, dass **immer nur genau ein Task oder Thread** gleichzeitig auf eine gemeinsame Ressource (z. B. Speicherbereich, Hardware-Register, Datei) zugreifen kann.

- **Prinzip:**
  - Nur **ein Besitzer gleichzeitig!**
  - Der Besitzer muss den Mutex **explizit wieder freigeben.**
  - Andere Tasks **warten blockierend**, bis der Mutex frei ist.

 © Christian Henke 2026

Semaphor

- **Zweck:**

  Ein *Semaphor* regelt **Zugriffszählungen** auf eine Ressource – also wie viele Tasks/POU‘s gleichzeitig hinein dürfen.

- **Prinzip:**
  - Hat einen **Zählerwert (N)**
  - Jeder Zugriff **dekrementiert** den Zähler.
  - Wenn Zähler = 0 → neue Zugriffe müssen warten.
  - Wenn eine Ressource freigegeben wird → **Zähler wird inkrementiert.**

 © Christian Henke 2026

Zusätzliche Dienste bzw. Programme auf der SPS

- Beispiele:
  - Webserver, Datenlogger, REST-API, OPC UA Server, E-Mail-Clients
- Auswirkungen:
  - Ggf. CPU-Jitter, RAM-Verbrauch, Latenzen
- Strategien:
  - Dienst-CPU-Affinity, Quality of Service-Management
  - Einstellbare Taktraten z.B. TwinCAT Core Boost

 © Christian Henke 2026